

实验六 跨平台综合项目实战

项目定位: 智慧农业大棚环境监测与智能控制系统

小组项目: 农业大棚监控系统 (AGMS)

个人岗位: Embedded开发 (图中第三个)

技术栈: C/C++ + STM32

主要功能: 下位机控制系统、传感器数据采集、自动灌溉控制、设备驱动开发、产品化：硬件稳定性、低功耗设计

服务对象: 农业大棚种植户、农场管理人员

核心目标: 实现大棚环境的实时监测、智能预警、远程控制与数据分析

项目周期: 14天 (2026年5月1日 - 2026年5月14日)

PDF导出说明: 各节ASCII示意图位于语言标记为 `text` 的fenced代码块内，框线中均为纯ASCII。

目录

- [1. 项目背景与目标](#)
- [2. 现状分析](#)
- [3. 功能设计](#)
- [4. 技术架构](#)
- [5. 创新亮点](#)
- [6. 对接方案](#)
- [7. Embedded端详细设计](#)
- [8. 实施计划](#)
- [9. 团队与分工](#)
- [10. 风险与对策](#)
- [附录](#)

1. 项目背景与目标

1.1 项目背景

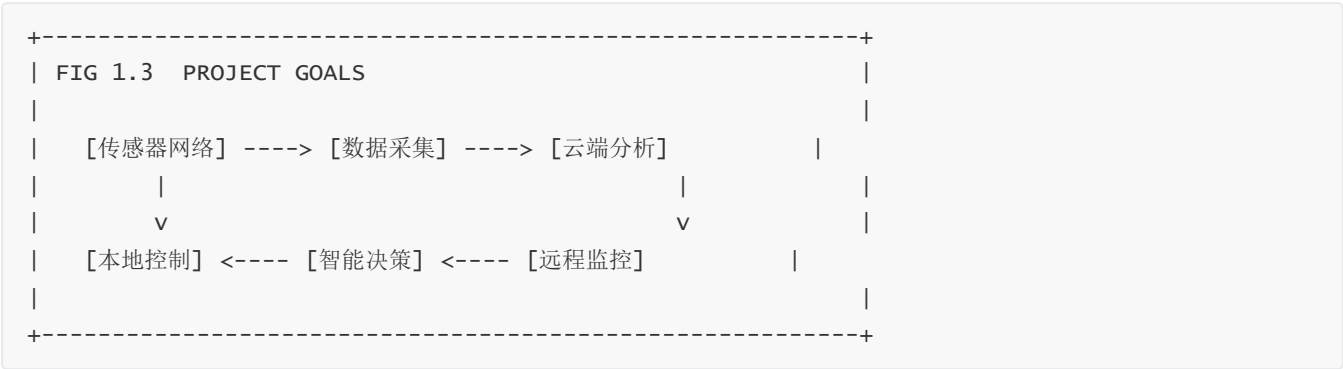
我国农业现代化进程加速，智慧农业成为农业转型升级的重要方向。传统农业大棚管理面临以下问题：

- 人工监测效率低：**依赖人工巡视，无法24小时实时监测
- 环境调控滞后：**发现问题后人工处理，错失最佳调控时机
- 数据缺乏积累：**历史数据无法有效利用，难以形成种植经验
- 人力成本高：**需要大量人工参与日常管理

1.2 现有痛点

痛点	现状	影响
监测不实时	人工定时巡查，间隔长	环境异常无法及时发现
调控不及时	发现问题后再人工干预	作物生长受影响
多棚管理难	大棚分散，管理困难	人力成本高，效率低
数据孤岛	各系统独立运行	无法进行综合分析
经验难传承	依赖个人经验	种植技术难以标准化

1.3 项目目标



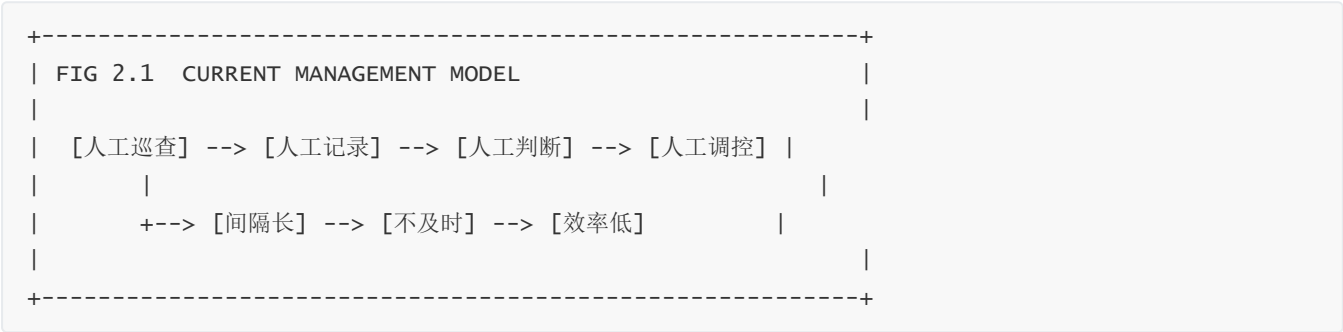
图注：从传感器数据采集、云端分析、远程监控到智能决策、本地控制的闭环。

1.4 核心价值

- **种植户**：实时掌握大棚环境，及时预警，降低损失
- **管理人员**：远程管理多棚，提升管理效率
- **技术专家**：数据分析支持，优化种植策略

2. 现状分析

2.1 现有农业大棚管理模式



图注：传统模式依赖人工，间隔长、不及时、效率低。

2.2 系统能力对比

功能	传统大棚	AGMS系统
监测频率	人工定时	7×24自动监测
响应速度	人工发现后处理	异常自动预警
数据记录	纸质/Excel	云端数据库存储
远程控制	需现场操作	手机APP远程控制
多棚管理	单棚管理	统一平台管理
数据分析	无	历史趋势分析
AI建议	无	环境调节建议

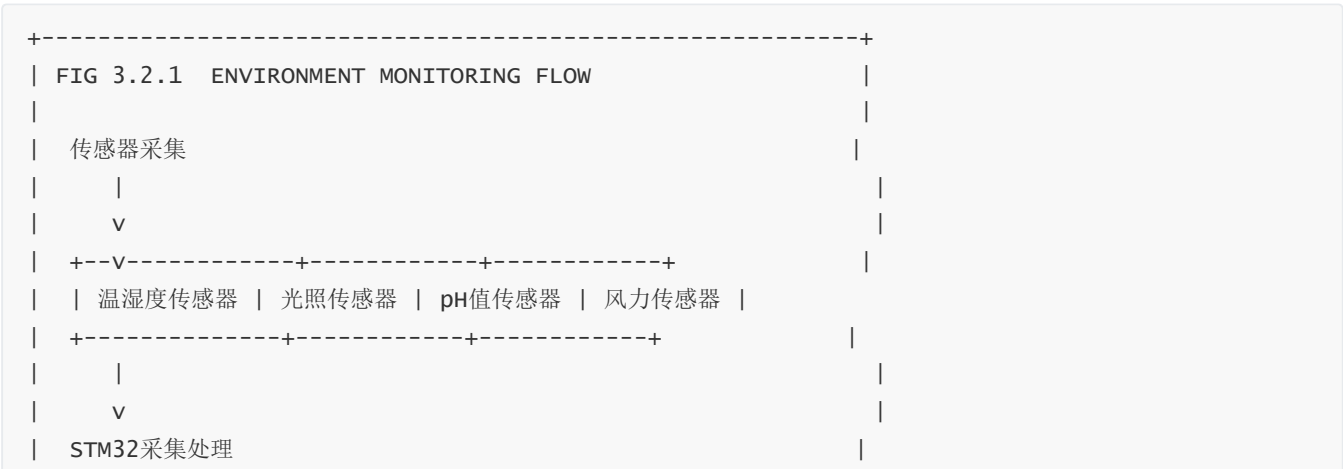
3. 功能设计

3.1 核心功能矩阵



3.2 重点功能详解

3.2.1 环境监测（核心功能）



√
数据上云/本地显示

监测指标：

- 温度：-20℃~60℃，精度±0.5℃
- 湿度：0%~100%RH，精度±3%
- 光照强度：0~200000 lux
- pH值：0~14，精度±0.1
- 风力等级：0~12级



3.2.2 智能预警 (创新亮点)



预警类型:

- 高温/低温预警
- 高湿/低湿预警
- 光照不足预警
- pH值异常预警
- 强风预警



3.2.3 远程控制



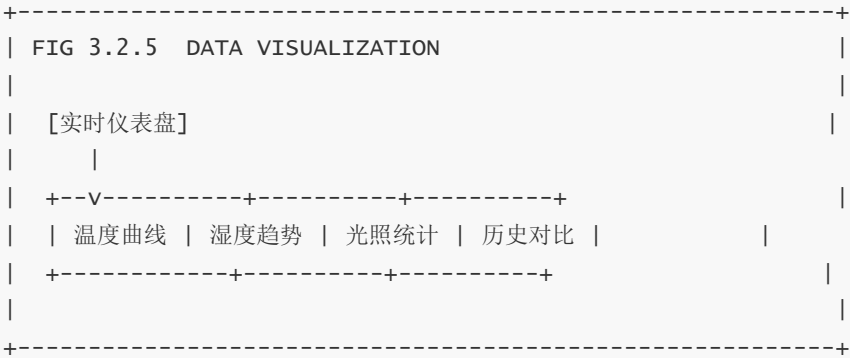
3.2.4 AI环境调节建议（创新亮点）

基于当前环境数据和作物类型，提供智能调控建议：

- 温度调节建议
- 湿度调节建议
- 光照补充建议

- ### ● 灌溉时机建议

3.2.5 数据可视化



3.3 多端功能分布

功能	Android端	HarmonyOS端	STM32端
实时数据展示	✓	✓	✓(本地)
历史数据查询	✓	✓	-
远程控制	✓	✓	-
智能预警	✓	✓	✓(本地报警)
AI建议	✓	✓	-
数据上传	-	-	✓
本地控制	-	-	✓

4. 技术架构

4.1 整体架构

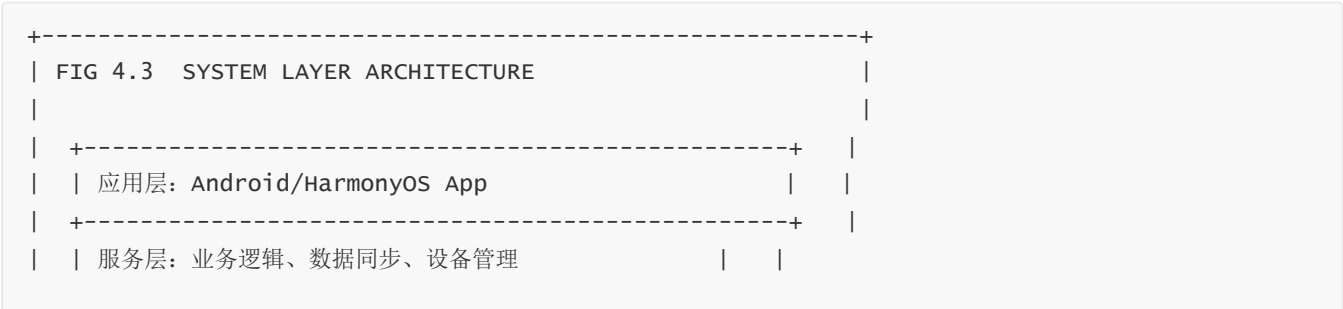


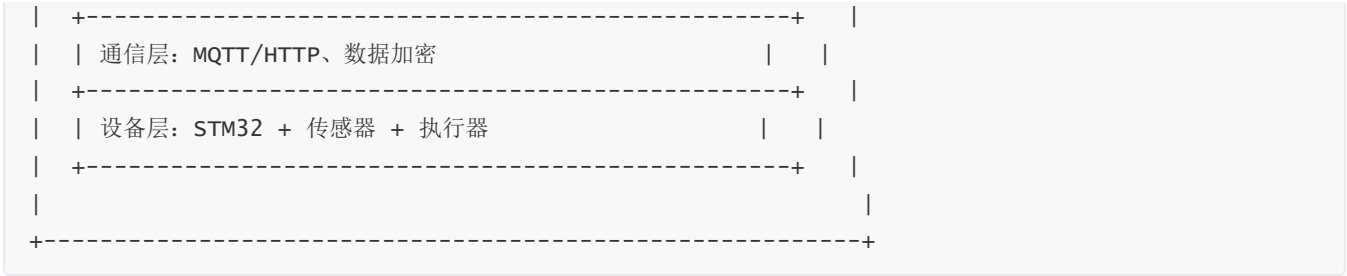
三层架构：Android/HarmonyOS App → 云端服务器 → STM32下位机 → 传感器/执行器
数据流：传感器采集→STM32→云端→App展示；控制流：App→云端→STM32→执行器

4.2 技术选型

层级	技术选型	说明
Android端	Kotlin + Android SDK	原生开发，性能优秀
HarmonyOS端	ArkTS + HarmonyOS SDK	分布式能力，多设备协同
嵌入式端	C/C++ + STM32	实时控制，稳定可靠
通信协议	MQTT/HTTP	轻量级物联网协议
数据库	SQLite + 云端数据库	本地缓存+云端存储
云平台	私有云/阿里云IoT	数据存储与分析

4.3 系统分层架构





5. 创新亮点

5.1 四大创新亮点



5.2 创新对比分析

创新点	传统方式	本项目	提升效果
监测方式	人工定时巡查	7×24自动监测	监测频率提升100倍
响应速度	人工发现处理	异常自动预警	响应时间从小时缩短到秒
数据利用	无数据积累	历史数据分析	支持种植策略优化
管理方式	单棚独立管理	多棚统一平台	管理效率提升300%

6. 对接方案

6.1 多端数据同步



	Android HarmonyOS	
	+-----+-----+	
+-----+		

6.2 设备对接方案

设备类型	对接方式	数据频率
温湿度传感器	数字接口(I2C/SPI)	每5秒
光照传感器	模拟/数字接口	每10秒
pH值传感器	模拟接口	每60秒
风力传感器	数字接口	每30秒
卷帘电机	继电器控制	按需
灌溉系统	电磁阀控制	按需

6.3 平台兼容性

- **Android端**: Android 8.0及以上
- **HarmonyOS端**: HarmonyOS 2.0及以上
- **嵌入式端**: STM32F103/STM32F407系列

7. Embedded端详细设计

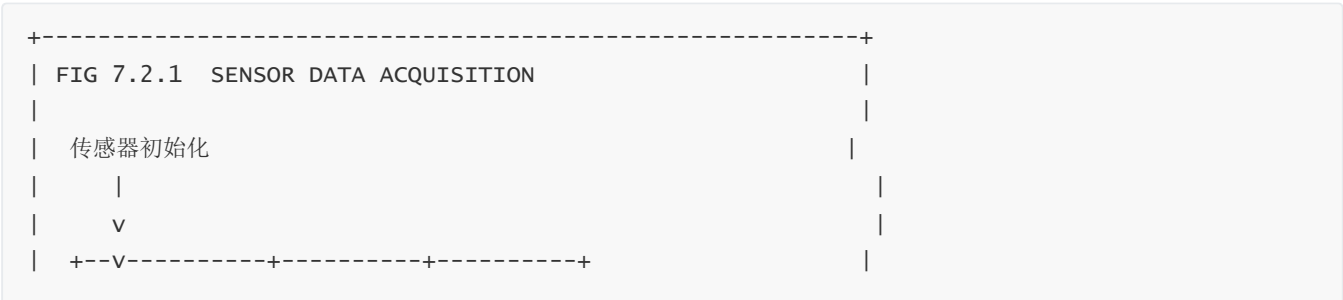
7.1 设计目标

在农业大棚监控系统中，Embedded端作为下位机控制系统，是整个系统的"神经中枢"，承担着：

- **数据采集**: 实时采集各类环境传感器数据
- **本地控制**: 根据环境参数自动控制大棚设备
- **通信中转**: 将采集数据上传云端，接收云端控制指令
- **边缘计算**: 本地进行阈值判断、异常预警

7.2 功能模块设计

7.2.1 传感器数据采集模块

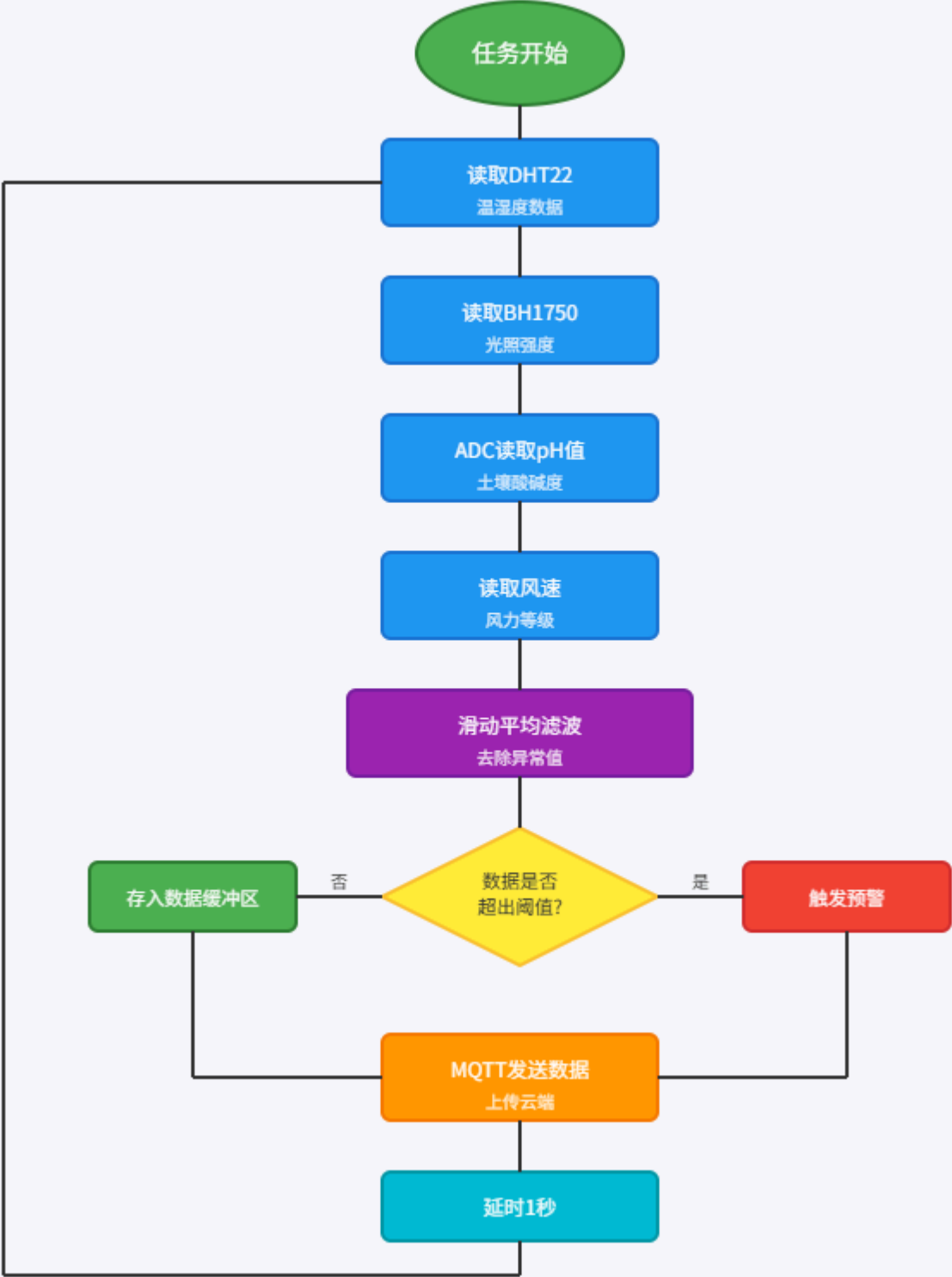


```

| | DHT22读取 | BH1750读取 | pH读取 | 风速读取 | |
| +-----+-----+-----+ |
| | | | | |
| | v | | |
| | 数据校验与滤波 | |
| | | | |
| | v | |
| | 存储到数据缓冲区 |
| |
+-----+

```

SensorTask 数据采集流程



读取DHT22/BH1750/pH/风速 → 滑动平均滤波 → 阈值判断 → 正常则上传/异常则报警

采集频率设计：

传感器	采集频率	接口类型
温湿度(DHT22)	5秒	单总线
光照(BH1750)	10秒	I2C

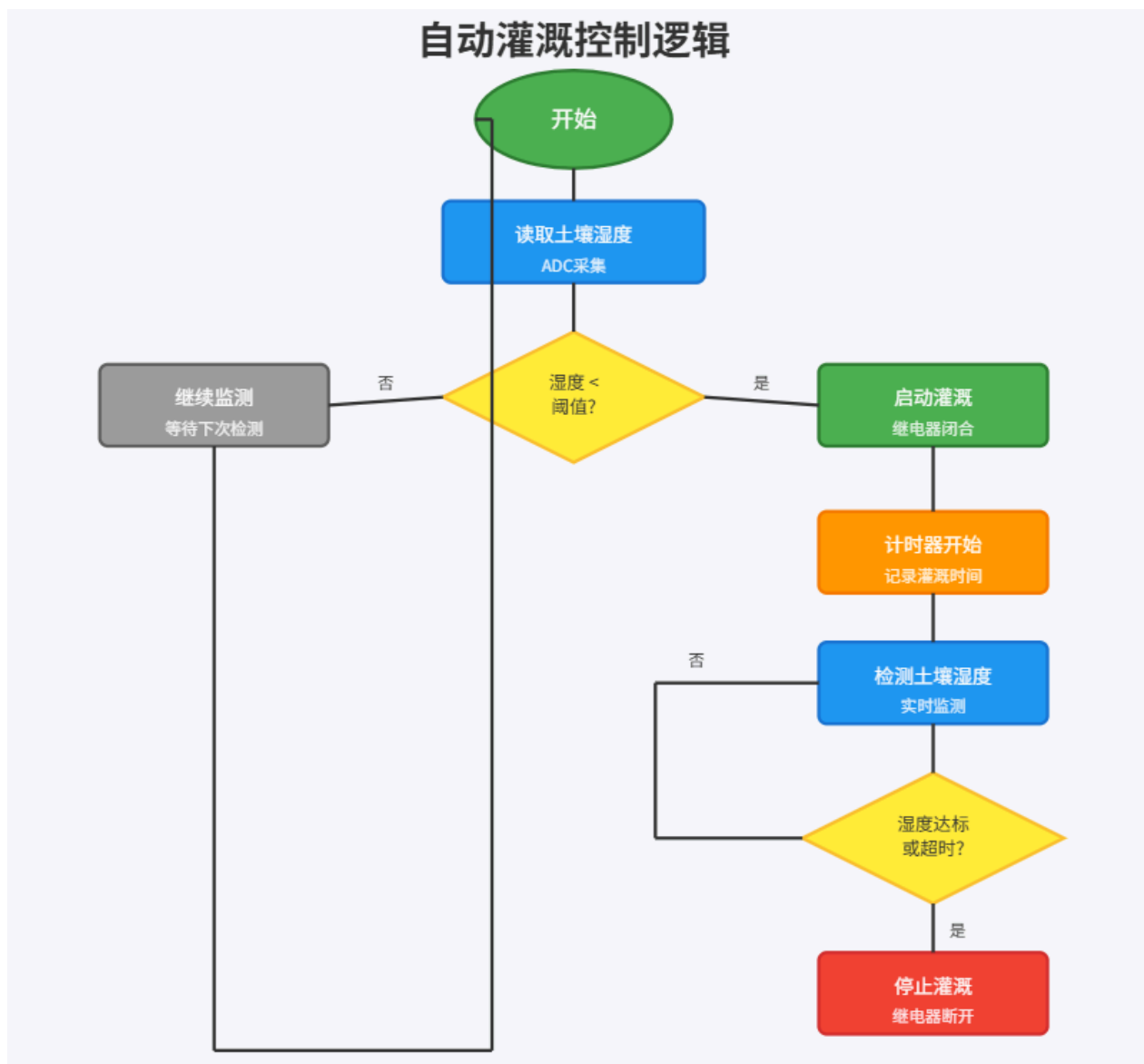
传感器	采集频率	接口类型
pH值	60秒	ADC
风速	30秒	数字输入

7.2.2 自动灌溉控制模块



控制策略：

- 阈值触发：土壤湿度低于设定值自动启动
- 定时灌溉：支持按时间段自动灌溉
- 手动控制：支持远程手动开启/关闭
- 安全保护：最大灌溉时间限制，防止溢出



读取土壤湿度 → 湿度<阈值? → 是: 启动灌溉+计时 → 湿度达标或超时 → 停止灌溉

7.2.3 环境调控模块

温度调控:

- 高温 (>35°C) : 开启通风、关闭补光
- 低温 (<10°C) : 关闭通风、开启补光

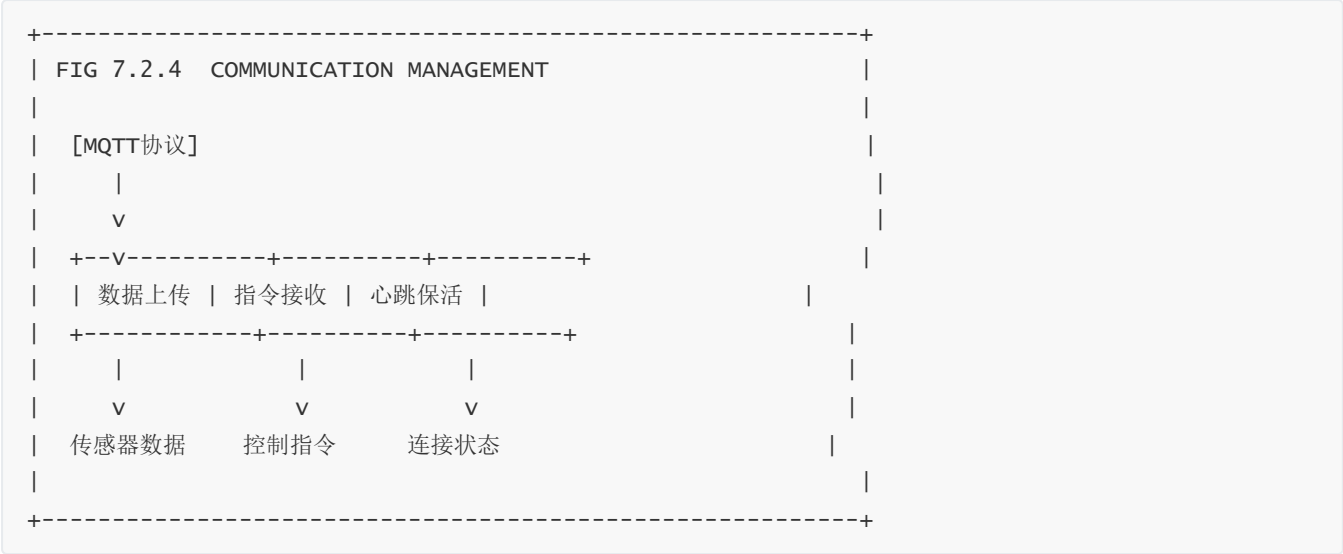
湿度调控:

- 高湿 (>90%) : 开启通风、除湿
- 低湿 (<40%) : 启动加湿、减少通风

光照调控:

- 光照不足: 开启补光灯
- 光照过强: 启动遮阳帘

7.2.4 通信管理模块



通信设计：

- 协议：MQTT over TCP/IP
- 数据格式：JSON
- 发布主题：greenhouse/data（上行）
- 订阅主题：greenhouse/control（下行）
- 心跳间隔：60秒

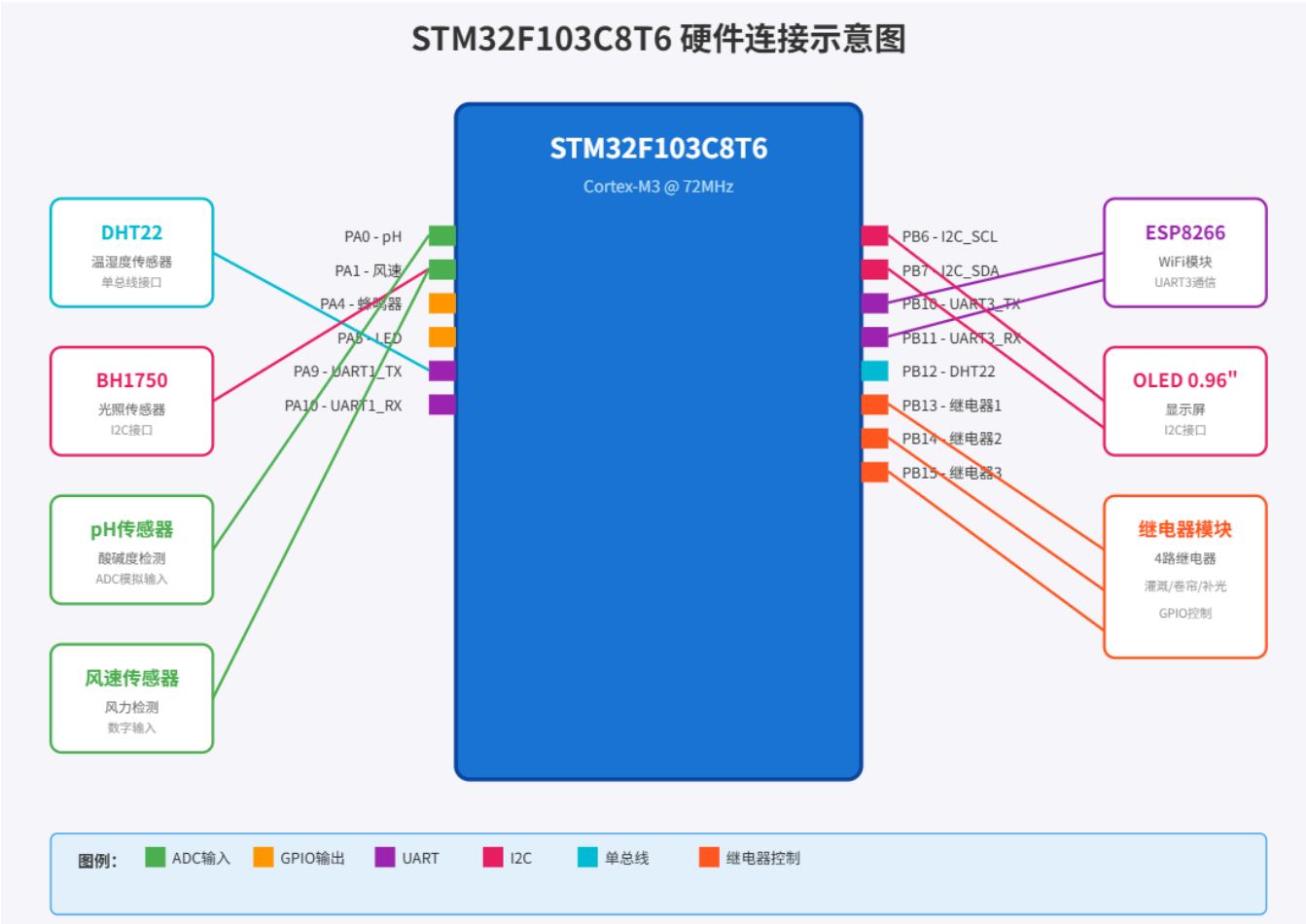
7.3 硬件设计方案

7.3.1 硬件清单

器件	型号	数量	说明
主控芯片	STM32F103C8T6	1	Cortex-M3, 72MHz
温湿度传感器	DHT22	2	温度-40~80℃, 湿度0-100%
光照传感器	BH1750	1	光照强度0-65535 lux
pH传感器	模拟pH计V1.1	1	pH范围0-14
风速传感器	数字风速仪	1	0-30m/s
WiFi模块	ESP8266-01S	1	MQTT通信
显示屏	OLED 0.96寸	1	I2C接口, 128x64
继电器模块	5V四路	2	控制灌溉/卷帘/补光
电源模块	5V/3.3V稳压	1	AMS1117-3.3
蜂鸣器	有源蜂鸣器	1	报警提示
LED指示灯	3mm彩色	4	状态指示

7.3.2 接口分配

+-----+ FIG 7.3.2 PIN ASSIGNMENT STM32F103C8T6 +-----+ PA0 - pH模拟输入 PA1 - 风速输入 PA4 - 蜂鸣器 PA5 - LED_STATUS PA9 - UART1_TX PA10 - UART1_RX PB6 - I2C_SCL PB7 - I2C_SDA PB10 - UART3_TX(ESP) PB11 - UART3_RX(ESP) PB12 - DHT22 PB13 - 继电器1(灌溉) PB14 - 继电器2(卷帘) PB15 - 继电器3(补光) +-----+ +-----+		
--	--	--



- PA0: pH传感器 (ADC)

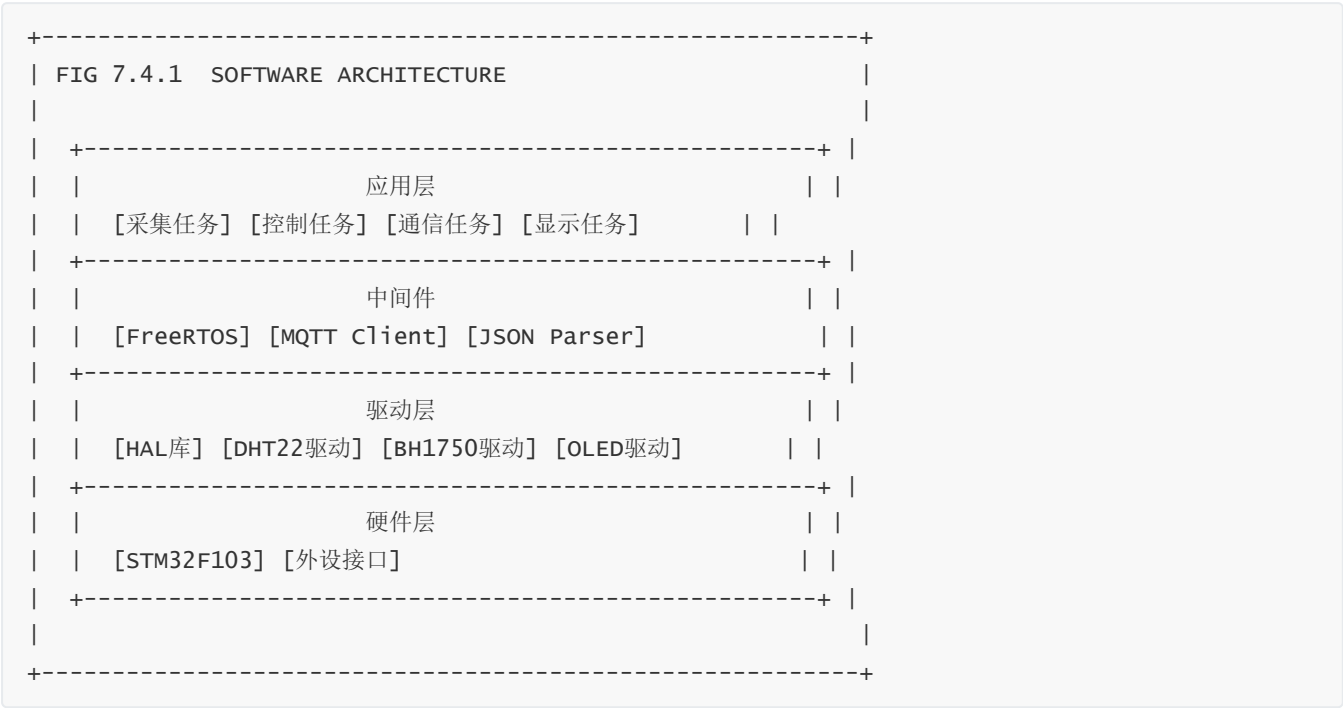
- PA1：风速传感器
- PA4/PA5：蜂鸣器/LED
- PB6/PB7：I2C（OLED、BH1750）
- PB10/PB11：UART3（ESP8266）
- PB12：DHT22
- PB13-PB15：继电器（灌溉/卷帘/补光）

7.3.3 电源设计

- 输入电压：5V DC（适配器/太阳能）
- 主控供电：3.3V（AMS1117稳压）
- 传感器供电：5V/3.3V可切换
- 功耗估算：
 - 运行模式：约150mA
 - 睡眠模式：约50mA

7.4 软件设计方案

7.4.1 软件架构

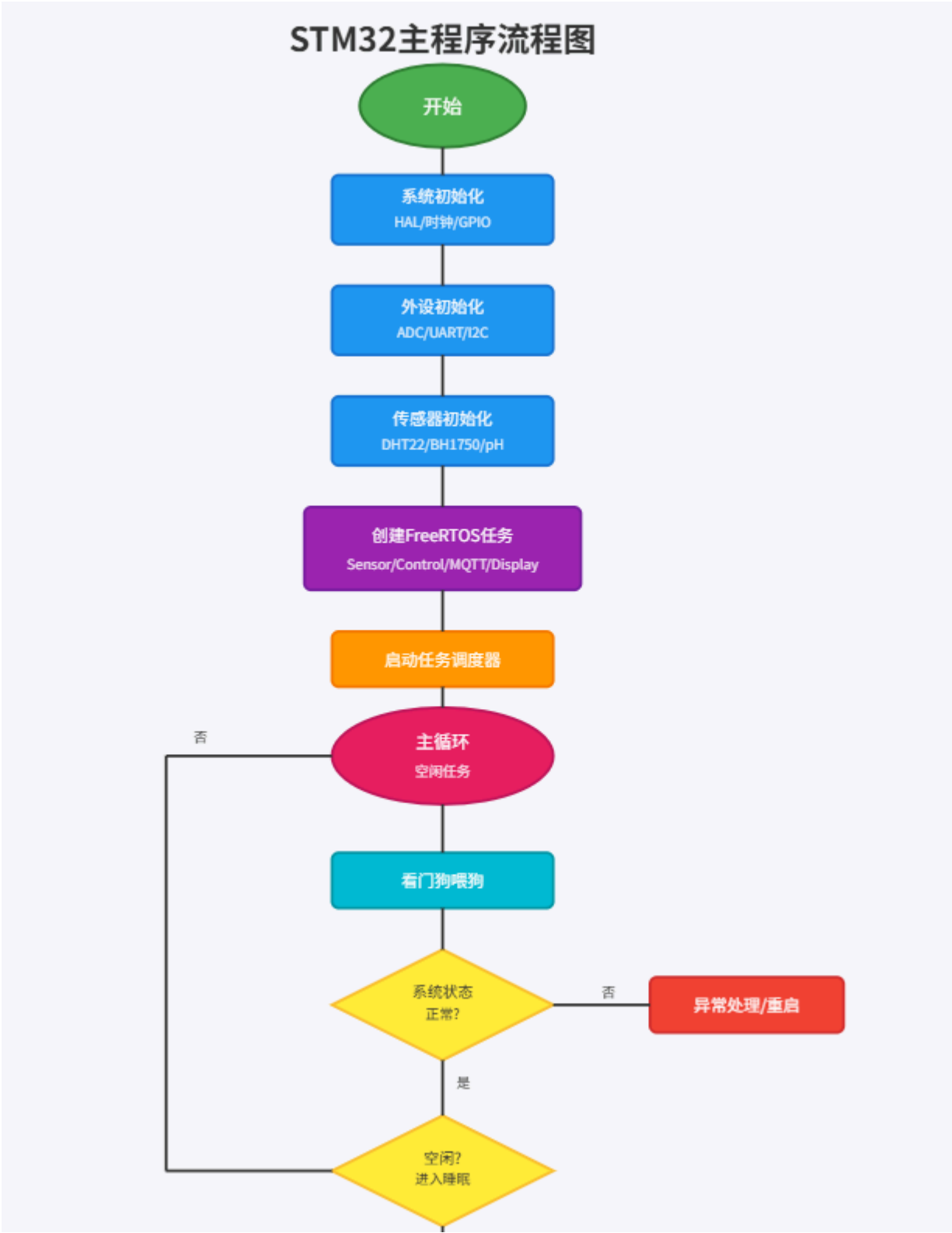


7.4.2 任务调度设计

使用FreeRTOS实现多任务调度：

任务	优先级	周期	功能
SensorTask	High	1s	传感器数据采集
ControlTask	High	2s	设备控制逻辑

任务	优先级	周期	功能
MqttTask	Normal	-	MQTT通信处理
DisplayTask	Low	1s	OLED显示更新
LedTask	Low	500ms	状态指示灯



系统初始化 → 外设初始化 → 传感器初始化 → 创建FreeRTOS任务 → 启动调度器 → 主循环（看门狗+状态检查）

7.4.3 核心代码框架

```
/* main.c - 主程序 */
#include "main.h"
#include "sensor.h"
#include "control.h"
#include "mqtt.h"
#include "display.h"

// FreeRTOS任务句柄
TaskHandle_t sensorTaskHandle = NULL;
TaskHandle_t controlTaskHandle = NULL;
TaskHandle_t mqttTaskHandle = NULL;
TaskHandle_t displayTaskHandle = NULL;

int main(void)
{
    // HAL初始化
    HAL_Init();
    SystemClock_Config();

    // 外设初始化
    MX_GPIO_Init();
    MX_ADC1_Init();
    MX_I2C1_Init();
    MX_USART1_UART_Init();
    MX_USART3_UART_Init();
    MX_TIM2_Init();

    // 模块初始化
    Sensor_Init();
    Control_Init();
    MQTT_Init();
    Display_Init();

    // 创建FreeRTOS任务
    xTaskCreate(SensorTask, "Sensor", 256, NULL, 3, &sensorTaskHandle);
    xTaskCreate(ControlTask, "Control", 256, NULL, 3, &controlTaskHandle);
    xTaskCreate(MqttTask, "MQTT", 512, NULL, 2, &mqttTaskHandle);
    xTaskCreate(DisplayTask, "Display", 256, NULL, 1, &displayTaskHandle);

    // 启动调度器
    vTaskStartScheduler();

    while (1);
}
```

7.4.4 关键算法

滑动平均滤波：

```
#define FILTER_SIZE 10

float MovingAverageFilter(float newValue, float *buffer, uint8_t *index)
{
    buffer[*index] = newValue;
    *index = (*index + 1) % FILTER_SIZE;

    float sum = 0;
    for (int i = 0; i < FILTER_SIZE; i++) {
        sum += buffer[i];
    }
    return sum / FILTER_SIZE;
}
```

阈值判断与预警：

```
typedef struct {
    float min;
    float max;
} Threshold_t;

Threshold_t tempThreshold = {10.0f, 35.0f};
Threshold_t humThreshold = {40.0f, 90.0f};

AlertLevel_t CheckThreshold(float value, Threshold_t *threshold)
{
    if (value < threshold->min) return ALERT_LOW;
    if (value > threshold->max) return ALERT_HIGH;
    return ALERT_NORMAL;
}
```

7.5 创新点 (Embedded端)

7.5.1 多传感器数据融合

- **滤波算法**：滑动平均滤波+中值滤波，去除异常值
- **数据校准**：温度补偿算法，提高测量精度
- **异常检测**：多传感器交叉验证，识别故障传感器

7.5.2 边缘智能控制

- **本地决策**：不依赖云端即可实现基础控制
- **快速响应**：本地控制响应时间<100ms
- **离线运行**：断网时可维持基本功能

7.5.3 低功耗设计

- **睡眠模式：**空闲时进入低功耗模式
- **动态调频：**根据负载动态调整CPU频率
- **外设管理：**不用时关闭外设电源

7.5.4 故障自诊断

- **看门狗机制：**程序异常自动重启
- **传感器检测：**自动识别传感器离线/故障
- **通信检测：**自动重连，异常上报

8. 实施计划

8.1 项目里程碑

阶段	时间	交付物
需求分析	D1-D2	PRD文档、技术方案
硬件搭建	D3-D5	传感器网络、STM32程序
应用开发	D6-D10	Android/HarmonyOS App
系统集成	D11-D12	联调测试、Bug修复
验收交付	D13-D14	测试报告、使用文档

8.2 详细任务分解

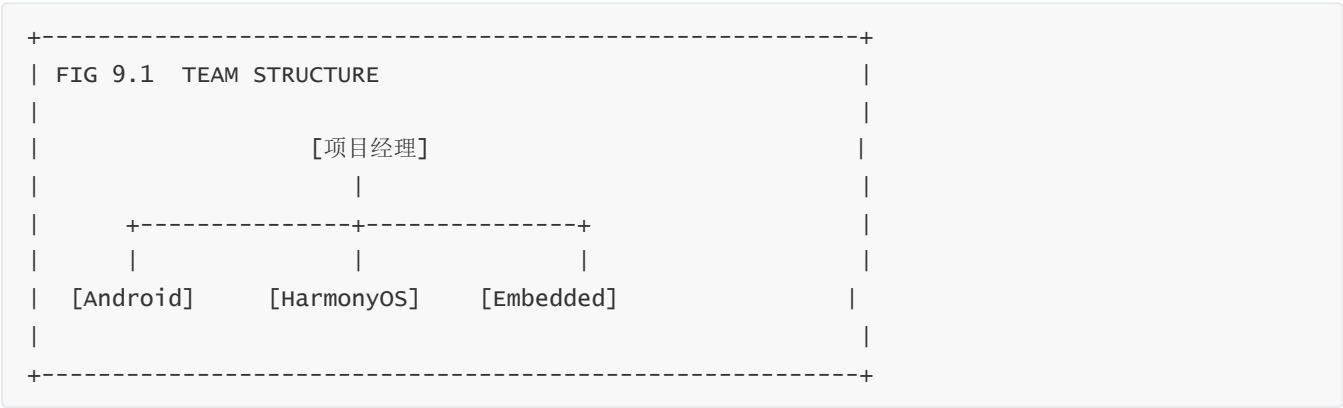
任务	负责人	截止时间	状态
需求文档编写	全员	D2	进行中
硬件选型采购	Embedded	D3	待开始
传感器驱动开发	Embedded	D5	待开始
Android App开发	Android	D10	待开始
HarmonyOS App开发	HarmonyOS	D10	待开始
云端服务搭建	全员	D7	待开始
系统集成测试	全员	D12	待开始
文档编写	全员	D14	待开始

8.3 Embedded端详细计划

任务	截止时间	状态	备注
硬件选型与采购	D3	待开始	确认传感器型号
STM32开发环境搭建	D3	待开始	Keil/VSCode+PlatformIO
基础外设驱动(GPIO/UART/I2C/ADC)	D4	待开始	HAL库配置
DHT22驱动开发	D5	待开始	单总线协议
BH1750驱动开发	D5	待开始	I2C通信
pH值采集模块	D6	待开始	ADC+校准
OLED显示驱动	D6	待开始	SSD1306
ESP8266 MQTT通信	D8	待开始	AT指令或SDK
继电器控制模块	D7	待开始	GPIO控制
自动灌溉逻辑	D8	待开始	阈值判断
环境调控逻辑	D9	待开始	多参数联动
FreeRTOS移植	D7	待开始	任务调度
低功耗优化	D12	待开始	睡眠模式
系统联调测试	D11	待开始	与App端对接

9. 团队与分工

9.1 团队结构



9.2 角色分工

角色	职责	技术栈
项目经理	需求管理、进度把控、文档编写	-
Android开发	Android App开发	Kotlin + Android SDK
HarmonyOS开发	HarmonyOS App开发	ArkTS + HarmonyOS SDK
Embedded开发	下位机开发、硬件调试	C/C++ + STM32

9.3 Embedded端详细分工

岗位名称：Embedded开发

技术栈：C/C++ + STM32

主要职责：

- 1. 下位机控制系统设计与实现
- 2. 传感器数据采集与处理
- 3. 自动灌溉控制逻辑开发
- 4. 设备驱动程序开发
- 5. 硬件稳定性优化
- 6. 低功耗设计实现

主要功能：

- 下位机控制系统
- 传感器数据采集（温湿度、光照、pH值、风力）
- 自动灌溉控制
- 设备驱动开发
- 产品化：硬件稳定性保障
- 产品化：低功耗设计

10. 风险与对策

10.1 风险识别

风险	等级	应对策略
硬件兼容性问题	高	提前选型测试，准备备用方案
通信稳定性问题	中	设计重连机制，本地缓存数据
开发进度延误	中	每日站会，关键路径优先

风险	等级	应对策略
跨端数据一致性	中	统一数据格式，严格测试验证

10.2 Embedded端特有风险

风险	等级	应对策略
传感器精度不达标	高	选型前测试，准备备用型号
ESP8266连接不稳定	中	添加自动重连机制，本地缓存
功耗过高	中	优化电源管理，必要时更换供电方案
实时性不足	低	优化任务优先级，关键任务提高优先级

10.3 质量保证

- 代码审查**：关键模块必须经过Code Review
- 测试覆盖**：单元测试+集成测试+系统测试
- 文档规范**：接口文档、使用手册齐全

10.4 测试计划

测试项	测试方法	通过标准
传感器精度测试	对比标准仪器	误差<±5%
稳定性测试	连续运行72小时	无死机、无重启
通信测试	模拟网络中断	自动重连成功
响应时间测试	发送控制指令	响应<500ms

附录

附录A. 接口清单

接口	方法	说明
/api/sensor/data	GET	获取传感器数据
/api/sensor/history	GET	获取历史数据
/api/control/command	POST	发送控制指令
/api/alert/config	POST	配置预警阈值

附录B. MQTT通信协议

数据上报格式：

```
{
  "device_id": "agms_001",
  "timestamp": "2026-05-12T10:30:00",
  "data": {
    "temperature": 25.5,
    "humidity": 68.0,
    "light": 12500,
    "ph": 6.8,
    "wind_speed": 2.5
  },
  "status": "normal"
}
```

控制指令格式：

```
{
  "command": "irrigation",
  "action": "start",
  "duration": 300
}
```

附录C. 硬件清单

设备	型号	数量
主控板	STM32F103C8T6	1
温湿度传感器	DHT22	2
光照传感器	BH1750	1
pH传感器	模拟pH计	1
风力传感器	数字风速仪	1
继电器模块	5V四路	2

附录D. 引脚定义表

功能	引脚	模式	说明
pH输入	PA0	ADC	模拟输入
风速输入	PA1	Input	数字输入
蜂鸣器	PA4	Output	高电平触发
LED状态	PA5	Output	高电平点亮

功能	引脚	模式	说明
UART1_TX	PA9	AF_PP	调试串口
UART1_RX	PA10	AF_PP	调试串口
I2C_SCL	PB6	AF_OD	OLED/BH1750
I2C_SDA	PB7	AF_OD	OLED/BH1750
UART3_TX	PB10	AF_PP	ESP8266
UART3_RX	PB11	AF_PP	ESP8266
DHT22	PB12	Input	单总线
继电器1	PB13	Output	灌溉
继电器2	PB14	Output	卷帘
继电器3	PB15	Output	补光

附录E. 参考资料

- STM32F103参考手册
- DHT22数据手册
- BH1750数据手册
- FreeRTOS官方文档
- MQTT协议规范

文档版本: v1.0
项目: 农业大棚监控系统(AGMS)
个人岗位: Embedded开发
技术栈: C/C++ + STM32
最后更新: 2026年5月